

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6.2
«Работа с БД в СУБД MongoDB»
по дисциплине «Проектирование и реализация баз данных»

БД «learn»

Обучающийся Дидык Глеб Олегович
Факультет прикладной информатики
Группа К3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии
Преподаватель Говорова Марина Михайловна, Белов Александр
Олегович

Санкт-Петербург
2026

1. Наименование работы

Лабораторная работа № 6.2 «Работа с БД в СУБД MongoDB».

2. Цель работы

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегацией и изменением данных, ссылками и индексами в базе данных MongoDB.

3. Практическое задание

В ходе лабораторной работы необходимо создать базу данных learn, заполнить коллекцию unicorns документами, выполнить запросы выборки, логические условия, работу с массивами, вложенными объектами, курсорами, агрегированными запросами, операциями изменения и удаления данных, ссылками между коллекциями, индексами и анализом плана запроса.

4. Выполнение работы

4.1. Практическое задание 2.1.1. Создание БД learn и заполнение коллекции unicorns

Команды:

```
use learn
```

```
db.unicorns.deleteMany({})
```

```
db.unicorns.insertMany([
```

```
  {name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63},
```

```
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
```

```
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
```

```
  {name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
```

```
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f',  
  vampires: 80},
```

```
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
```

```
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
```

```
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
```

```
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
```

```
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
```

```
{name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}  
])
```

```
document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm',  
vampires: 165}
```

```
db.unicorns.insertOne(document)
```

```
db.unicorns.find()
```

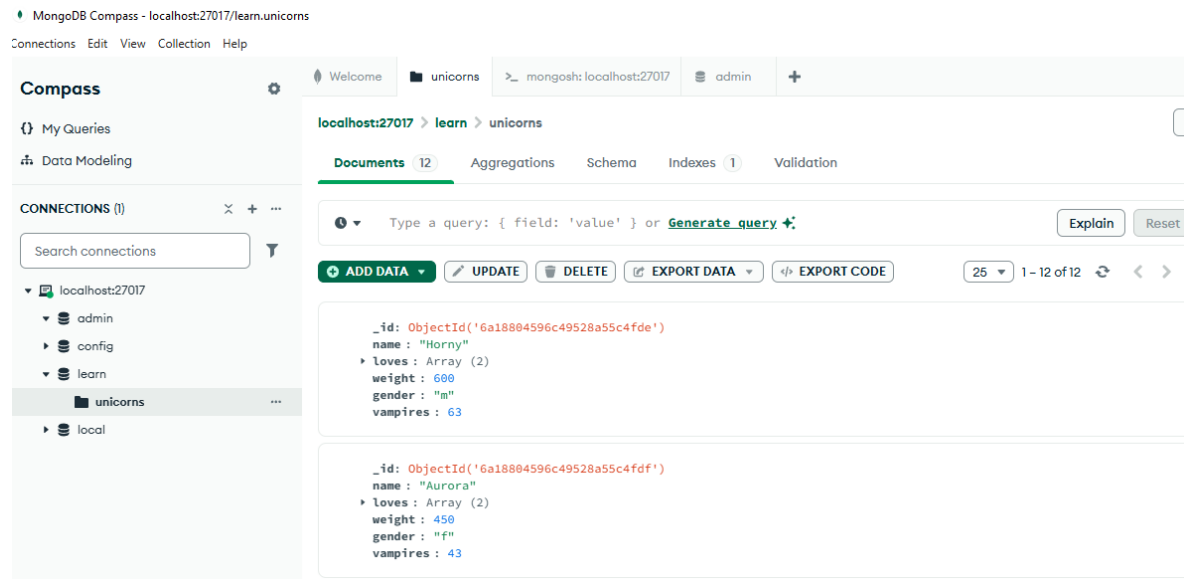


Рисунок 1 - Создание базы данных learn и заполнение коллекции unicorns

Вывод: В результате создана база данных learn, коллекция unicorns содержит 12 документов.

4.2. Практическое задание 2.2.1. Выборка самцов и самок

Команды:

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
```

```
> db.unicorns.find({gender: 'm'}).sort({name: 1})
< {
  _id: ObjectId('6a18805b96c49528a55c4fe9'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a18804596c49528a55c4fde'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  _id: ObjectId('6a18804596c49528a55c4fe4'),
```

Рисунок 2 - Выборка самцов единорогов с сортировкой по имени

```

> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
< {
  _id: ObjectId('6a18804596c49528a55c4fdf'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a18804596c49528a55c4fe3'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 733,
  gender: 'f',
  vampires: 40
}
{
  _id: ObjectId('6a18804596c49528a55c4fe6'),

```

Рисунок 3 - Выборка первых трех самок единорогов с сортировкой по имени

Вывод: Запросы позволяют получить список самцов и ограниченный список первых трех самок с сортировкой по имени.

4.3. Практическое задание 2.2.1. Поиск самок, предпочитающих carrot

Команды:

```
db.unicorns.find({gender: 'f', loves: 'carrot'})
```

```
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
```

```
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
```

```
> db.unicorns.find({gender: 'f', loves: 'carrot'})
< {
  _id: ObjectId('6a18804596c49528a55c4fdf'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a18804596c49528a55c4fe2'),
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  _id: ObjectId('6a18804596c49528a55c4fe8'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
```

Рисунок 4 - Поиск всех самок единорогов, предпочитающих carrot

```
> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
< {
  _id: ObjectId('6a18804596c49528a55c4fdf'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Рисунок 5 - Ограничение выборки с помощью findOne

```
> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
< {
  _id: ObjectId('6a18804596c49528a55c4fdf'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Рисунок 6 - Ограничение выборки с помощью limit

Вывод: Методы find, findOne и limit позволяют получить все подходящие документы или ограничить выдачу одним документом.

4.4. Практическое задание 2.2.2. Исключение полей из результата

Команды:

```
db.unicorns.find(
  {gender: 'm'},
  {loves: 0, gender: 0}
).sort({name: 1})
```

```

> db.unicorns.find(
  {gender: 'm'},
  {loves: 0, gender: 0}
).sort({name: 1})
< {
  _id: ObjectId('6a18805b96c49528a55c4fe9'),
  name: 'Dunx',
  weight: 704,
  vampires: 165
}
{
  _id: ObjectId('6a18804596c49528a55c4fde'),
  name: 'Horny',
  weight: 600,
  vampires: 63
}
{
  _id: ObjectId('6a18804596c49528a55c4fe4'),
  name: 'Kenny',
  weight: 690,
  vampires: 39
}
{
  _id: ObjectId('6a18804596c49528a55c4fe7'),
  name: 'Pilot',
  weight: 650,
  vampires: 54
}
{
  _id: ObjectId('6a18804596c49528a55c4fe5'),
  name: 'RaLeigh',
  weight: 421,
  vampires: 2
}
{
  _id: ObjectId('6a18804596c49528a55c4fe1'),
  name: 'Roooooodles',
  weight: 575,
  vampires: 99
}
{
  _id: ObjectId('6a18804596c49528a55c4fe0'),
  name: 'Unicrom',
  weight: 984,
  vampires: 182
}

```

Рисунок 7 - Вывод списка самцов без полей loves и gender

Вывод: В результате у документов отсутствуют поля loves и gender, но остаются name, weight и vampires.

4.5. Практическое задание 2.2.3. Обратный порядок добавления

Команды:

```
db.unicorns.find().sort({$natural: -1})
```



```

> db.unicorns.find().sort({$natural: -1})
< {
  _id: ObjectId('6a18805b96c49528a55c4fe9'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a18804596c49528a55c4fe8'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
{
  _id: ObjectId('6a18804596c49528a55c4fe7'),
  name: 'Pilot',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 650,
  gender: 'm',
  vampires: 54
}
{
  _id: ObjectId('6a18804596c49528a55c4fe6'),
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}
{
  _id: ObjectId('6a18804596c49528a55c4fe5'),
  name: 'Raleigh',
  loves: [
    'apple',

```

Рисунок 8 - Вывод списка единорогов в обратном порядке добавления

Вывод: Параметр \$natural со значением -1 выводит документы в обратном порядке добавления.

4.6. Практическое задание 2.1.4. Первый элемент массива loves

Команды:

```

db.unicorns.find(
  {},
  {_id: 0, name: 1, loves: {$slice: 1}}
)

```

```
> db.unicorns.find(
  {},
  {_id: 0, name: 1, loves: {$slice: 1}}
)
< {
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Aurora',
  loves: [
    'carrot'
  ]
}
{
  name: 'Unicrom',
  loves: [
    'energon'
  ]
}
{
  name: 'Rooooooodles',
  loves: [
    'apple'
  ]
}
{
  name: 'Solnara',
  loves: [
    'apple'
  ]
}
{
  name: 'Ayna',
```

Рисунок 9 - Вывод первого предпочтения единорогов без идентификатора

Вывод: Оператор \$slice выводит только первый элемент массива loves, поле _id исключено.

4.7. Практическое задание 2.3.1. Самки весом от 500 до 700 кг

Команды:

```
db.unicorns.find(  
  {gender: 'f', weight: {$gte: 500, $lte: 700}},  
  {_id: 0}  
)
```

```
> db.unicorns.find(  
  
  {gender: 'f', weight: {$gte: 500, $lte: 700}},  
  
  {_id: 0}  
  
)  
< {  
  name: 'Solnara',  
  loves: [  
    'apple',  
    'carrot',  
    'chocolate'  
  ],  
  weight: 550,  
  gender: 'f',  
  vampires: 80  
}  
{  
  name: 'Leia',  
  loves: [  
    'apple',  
    'watermelon'  
  ],  
  weight: 601,  
  gender: 'f',  
  vampires: 33  
}  
{  
  name: 'Nimue',  
  loves: [  
    'grape',  
    'carrot'  
  ],  
  weight: 540,  
  gender: 'f'  
}
```

Рисунок 10 - Выборка самок единорогов весом от 500 до 700 кг

Вывод: Используются операторы сравнения \$gte и \$lte.

4.8. Практическое задание 2.3.2. Самцы весом от 500 кг, предпочитающие grape и lemon

Команды:

```
db.unicorns.find(  
  {  
    gender: 'm',  
    weight: {$gte: 500},  
    loves: {$all: ['grape', 'lemon']}  
  },  
  {_id: 0}  
)
```

```

> db.unicorns.find(

  {

    gender: 'm',

    weight: {$gte: 500},

    loves: {$all: ['grape', 'lemon']}}

  },

  {_id: 0}

)
< {
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}

```

Рисунок 11 - Выборка самцов весом от 500 кг, предпочитающих grape и lemon

Вывод: Оператор \$all проверяет, что массив loves содержит оба значения: grape и lemon.

4.9. Практическое задание 2.3.3. Документы без поля vampires

Команды:

```

db.unicorns.find(
  {vampires: {$exists: false}},
  {_id: 0}
)

```

```
> db.unicorns.find(
    {vampires: {$exists: false}},
    {_id: 0}
)
< {
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
```

Рисунок 12 - Поиск единорогов, не имеющих поля vampires

Вывод: Оператор \$exists:false нашел документ Nimue, в котором поле vampires отсутствует.

4.10. Практическое задание 2.3.4. Самцы с первым предпочтением

Команды:

```
db.unicorns.find(
  {gender: 'm'},
  {_id: 0, name: 1, loves: {$slice: 1}}
).sort({name: 1})
```

```
> db.unicorns.find(
  {gender: 'm'},
  {_id: 0, name: 1, loves: {$slice: 1}}
).sort({name: 1})
< {
  name: 'Dunx',
  loves: [
    'grape'
  ]
}
{
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Kenny',
  loves: [
    'grape'
  ]
}
{
  name: 'Pilot',
  loves: [
    'apple'
  ]
}
{
  name: 'Raleigh',
  loves: [
    'apple'
  ]
}
{
  name: 'Rooooooodles',
  loves: [
    'apple'
  ]
}
{
  name: 'Unicrom',
  loves: [
    'energon'
  ]
}
```

Рисунок 13 - Упорядоченный список самцов единорогов с первым предпочтением

Вывод: Выводятся имена самцов и первый элемент массива loves, документы отсортированы по имени.

4.11. Практическое задание 3.1.1. Создание towns с вложенным объектом mayor

Команды:

```
db.towns.drop()
```

```
db.towns.insertMany([
```

```
  {name: 'Punxsutawney', population: 6200, last_sensus: ISODate('2008-01-31'),  
  famous_for: ['phil the groundhog'], mayor: {name: 'Jim Wehrle'}}],
```

```
  {name: 'New York', population: 22200000, last_sensus: ISODate('2009-07-31'),  
  famous_for: ['statue of liberty', 'food'], mayor: {name: 'Michael Bloomberg', party:  
  'T'}}],
```

```
  {name: 'Portland', population: 528000, last_sensus: ISODate('2009-07-20'),  
  famous_for: ['beer', 'food'], mayor: {name: 'Sam Adams', party: 'D'}}
```

```
])
```

```
db.towns.find()
```



```

< {
  _id: ObjectId('6a18827e96c49528a55c4fea'),
  name: 'Punxsutawney',
  population: 6200,
  last_sensus: 2008-01-31T00:00:00.000Z,
  famous_for: [
    'phil the groundhog'
  ],
  mayor: {
    name: 'Jim Wehrle'
  }
}
{
  _id: ObjectId('6a18827e96c49528a55c4feb'),
  name: 'New York',
  population: 22200000,
  last_sensus: 2009-07-31T00:00:00.000Z,
  famous_for: [
    'statue of liberty',
    'food'
  ],
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
{
  _id: ObjectId('6a18827e96c49528a55c4fec'),
  name: 'Portland',
  population: 528000,
  last_sensus: 2009-07-20T00:00:00.000Z,
  famous_for: [
    'beer',
    'food'
  ],
  mayor: {
    name: 'Sam Adams',
    party: 'D'
  }
}
learn>

```

Рисунок 14 - Создание коллекции towns с вложенным объектом mayor

Вывод: Документы коллекции towns содержат вложенный объект mayor.

4.12. Практическое задание 3.1.1. Запросы к вложенному объекту mayor

Команды:

```
db.towns.find(  
  {'mayor.party': 'I'},  
  {_id: 0, name: 1, mayor: 1}  
)
```

```
db.towns.find(  
  {'mayor.party': {'$exists: false'}},  
  {_id: 0, name: 1, mayor: 1}  
)
```



```
> db.towns.find(  
  
  {"mayor.party": "I"},  
  
  {_id: 0, name: 1, mayor: 1}  
  
  )  
< {  
  name: 'New York',  
  mayor: {  
    name: 'Michael Bloomberg',  
    party: 'I'  
  }  
}
```

Рисунок 15 - Поиск города с независимым мэром через поле mayor.party

```
> db.towns.find(

  {"mayor.party": {$exists: false}},

  {_id: 0, name: 1, mayor: 1}

)
< {
  name: 'Punxsutawney',
  mayor: {
    name: 'Jim Wehrle'
  }
}
learn> |
```

Рисунок 16 - Поиск города с мэром без поля party

Вывод: Оператор точка позволяет обращаться к полям вложенного документа mayor.

4.13. Практическое задание 3.1.2. JavaScript-функция и курсор

Команды:

```
var cursor = db.unicorns.find({
  $where: function() {
    return this.gender == 'm';
  }
}).sort({name: 1}).limit(2);
```

```
cursor.forEach(function(obj) {  
  print(obj.name + ' - ' + obj.gender + ' - ' + obj.weight);  
});
```

```
> var cursor = db.unicorns.find({  
  
  $where: function() {  
  
    return this.gender == 'm';  
  
  }  
  
}).sort({name: 1}).limit(2);  
  
cursor.forEach(function(obj) {  
  
  print(obj.name + " - " + obj.gender + " - " + obj.weight);  
  
});  
< Dunx - m - 704  
< Horny - m - 600  
learn>
```

Рисунок 17 - Использование JavaScript-функции и курсора для вывода первых двух самцов

Вывод: В актуальной версии MongoDB функция применена через \$where, так как find(fn) больше не поддерживается.

4.14. Практическое задание 3.2.1. Count

Команды:

```
db.unicorns.find({  
  gender: 'f',  
  weight: {$gte: 500, $lte: 600}  
}).count()
```

```
> db.unicorns.find({  
  
  gender: 'f',  
  
  weight: {$gte: 500, $lte: 600}  
  
}).count()  
< 2
```

Рисунок 18 - Подсчет количества самок единорогов весом от 500 до 600 кг
Вывод: Получено значение 2: подходят Solnara и Nimue.

4.15. Практическое задание 3.2.2. Distinct

Команды:

```
db.unicorns.distinct('loves')
```

```
> db.unicorns.distinct("loves")  
< [  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'papaya',      'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]
```

Рисунок 19 - Вывод уникального списка предпочтений единорогов

Вывод: Метод distinct вернул уникальные значения массива loves.

4.16. Практическое задание 3.2.3. Aggregate

Команды:

```
db.unicorns.aggregate([
  {
    $group: {
      _id: '$gender',
      count: {$sum: 1}
    }
  }
])
```

```
> db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      count: {$sum: 1}
    }
  }
])
< {
  _id: 'm',
  count: 7
}
{
  _id: 'f',
  count: 5
}
```

Рисунок 20 - Подсчет количества единорогов по полу с помощью aggregate

Вывод: С помощью \$group посчитано количество особей обоих полов: 7 самцов и 5 самок.

4.17. Практическое задание 3.3.1. Добавление Barny

Команды:

```
db.unicorns.insertOne({
  name: 'Barny',
  loves: ['grape'],
```

```
weight: 340,  
gender: 'm'  
}))
```

```
db.unicorns.find({name: 'Barney'}, {_id: 0})
```



```
> db.unicorns.insertOne({  
  
  name: 'Barney',  
  
  loves: ['grape'],  
  
  weight: 340,  
  
  gender: 'm'  
  
})  
  
db.unicorns.find({name: 'Barney'}, {_id: 0})  
< {  
  name: 'Barney',  
  loves: [  
    'grape'  
  ],  
  weight: 340,  
  gender: 'm'  
}
```

Рисунок 21 - Добавление единорога Barney и проверка содержимого коллекции

Вывод: В новой версии MongoDB использован insertOne как актуальный аналог добавления документа.

4.18. Практическое задание 3.3.2. Изменение Аюна

Команды:

```
db.unicorns.updateOne(
```



```
{name: 'Ayna'},  
{$set: {weight: 800, vampires: 51}}  
)
```

```
db.unicorns.find({name: 'Ayna'}, {_id: 0})
```

```
> db.unicorns.updateOne(  
  
  {name: 'Ayna'},  
  
  {$set: {weight: 800, vampires: 51}}  
  
)  
  
db.unicorns.find(  
  
  {name: 'Ayna'},  
  
  {_id: 0}  
  
)  
< {  
  name: 'Ayna',  
  loves: [  
    'strawberry',  
    'lemon'  
  ],  
  weight: 800,  
  gender: 'f',  
  vampires: 51  
}
```

Рисунок 22 - Изменение веса и количества vampires для единорога Аупа

Вывод: Оператор \$set изменил значения weight и vampires.

4.19. Практическое задание 3.3.3. Изменение Raleigh

Команды:

```
db.unicorns.updateOne(  
  {name: 'Raleigh'},  
  {$set: {loves: ['redbull']}}  
)
```

```
db.unicorns.find({name: 'Raleigh'}, {_id: 0})
```

```
> db.unicorns.updateOne(  
  
  {name: 'Raleigh'},  
  
  {$set: {loves: ['redbull']}}  
  
)  
  
db.unicorns.find(  
  
  {name: 'Raleigh'},  
  
  {_id: 0}  
  
)  
< {  
  name: 'Raleigh',  
  loves: [  
    'redbull'  
  ],  
  weight: 421,  
  gender: 'm',  
  vampires: 7  
}
```

Рисунок 23 - Изменение предпочтений единорога Raleigh

Вывод: У Raleigh массив loves заменен на значение redbull.

4.20. Практическое задание 3.3.4. Увеличение vampires у самцов

Команды:

```
db.unicorns.updateMany(  
  {gender: 'm'},  
  {$inc: {vampires: 5}}  
)
```

```
db.unicorns.find(  
  {gender: 'm'},  
  {_id: 0, name: 1, gender: 1, vampires: 1}  
).sort({name: 1})
```

```

> db.unicorns.updateMany(

  {gender: 'm'},

  {$inc: {vampires: 5}}

)

db.unicorns.find(

  {gender: 'm'},

  {_id: 0, name: 1, gender: 1, vampires: 1}

).sort({name: 1})
< {
  name: 'Barny',
  gender: 'm',
  vampires: 5
}
{
  name: 'Dunx',
  gender: 'm',
  vampires: 170
}
{
  name: 'Horny',
  gender: 'm',
  vampires: 68
}
{
  name: 'Kenny',
  gender: 'm',
  vampires: 44
}
{
  name: 'Pilot',
  gender: 'm',
  vampires: 59
}
{
  name: 'Raleigh',
  gender: 'm',
  vampires: 7
}
{
  name: 'Roooooodles',

```

Рисунок 24 - Увеличение количества vampires у всех самцов единорогов

Вывод: Оператор \$inc увеличил числовое поле vampires у всех документов, где gender равно m.

4.21. Практическое задание 3.3.5. Удаление mayor.party у Portland

Команды:

```
db.towns.updateOne(  
  {name: 'Portland'},  
  {$unset: {'mayor.party': 1}}  
)
```

```
db.towns.find(  
  {name: 'Portland'},  
  {_id: 0, name: 1, mayor: 1}  
)
```

```

> db.towns.updateOne(
  {name: 'Portland'},
  {$unset: {"mayor.party": 1}}
)

db.towns.find(
  {name: 'Portland'},
  {_id: 0, name: 1, mayor: 1}
)
< {
  name: 'Portland',
  mayor: {
    name: 'Sam Adams'
  }
}

```

Рисунок 25 - Изменение информации о городе Portland: удаление поля mayor.party
 Вывод: Оператор \$unset удалил поле party у вложенного объекта mayor.

4.22. Практическое задание 3.3.6. Добавление chocolate для Pilot

Команды:

```

db.unicorns.updateOne(
  {name: 'Pilot'},
  {$push: {loves: 'chocolate'}}
)

```

```
db.unicorns.find({name: 'Pilot'}, {_id: 0, name: 1, loves: 1})
```

```
> db.unicorns.updateOne(
  {name: 'Pilot'},
  {$push: {loves: 'chocolate'}}
)

db.unicorns.find(
  {name: 'Pilot'},
  {_id: 0, name: 1, loves: 1}
)
< {
  name: 'Pilot',
  loves: [
    'apple',
    'watermelon',
    'chocolate'
  ]
}
```

Рисунок 26 - Добавление chocolate в список предпочтений единорога Pilot

Вывод: Оператор \$push добавил новое значение в массив loves.

4.23. Практическое задание 3.3.7. Добавление sugar и lemon для Aurora

Команды:

```
db.unicorns.updateOne(
  {name: 'Aurora'},
  {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
```

)

```
db.unicorns.find({name: 'Aurora'}, {_id: 0, name: 1, loves: 1})
```

```
> db.unicorns.updateOne(
  {name: 'Aurora'},
  {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
)

db.unicorns.find(
  {name: 'Aurora'},
  {_id: 0, name: 1, loves: 1}
)
< {
  name: 'Aurora',
  loves: [
    'carrot',
    'grape',
    'sugar',
    'lemon'
  ]
}
```

Рисунок 27 - Добавление sugar и lemon в список предпочтений единорога Aurora

Вывод: Оператор \$addToSet добавил значения sugar и lemon без создания дублей.

4.24. Практическое задание 3.4.1. Удаление городов с беспартийными мэрами

Команды:

```
db.towns.deleteMany(
```



```
{'mayor.party': {$exists: false}}  
)
```

```
db.towns.find()
```

```
> db.towns.deleteMany(  
  
  {"mayor.party": {$exists: false}}  
  
)  
  
db.towns.find()  
< {  
  _id: ObjectId('6a18827e96c49528a55c4feb'),  
  name: 'New York',  
  population: 22200000,  
  last_sensus: 2009-07-31T00:00:00.000Z,  
  famous_for: [  
    'statue of liberty',  
    'food'  
  ],  
  mayor: {  
    name: 'Michael Bloomberg',  
    party: 'I'  
  }  
}
```

Рисунок 28 - Удаление городов с беспартийными мэрами из коллекции towns

Вывод: После удаления в коллекции towns остался город New York, у которого mayor.party равен I.

4.25. Практическое задание 3.4.1. Очистка коллекции towns и просмотр коллекций

Команды для выполнения:

```
db.towns.deleteMany({})
```

```
db.towns.countDocuments()
```

```
db.getCollectionNames()
```



```
> db.towns.deleteMany({})
< {
  acknowledged: true,
  deletedCount: 1
}
> db.towns.countDocuments()
< 0
> db.getCollectionNames()
< [ 'unicorns', 'towns' ]
```

Рисунок 29 - Очистка коллекции towns и просмотр списка коллекций

Вывод: после очистки в towns не осталось документов, список коллекций подтверждает наличие рабочих коллекций базы learn.

4.26. Практическое задание 4.1.1. Ссылки на зоны обитания

Команды для выполнения:

```
db.habitats.drop()
```

```
db.habitats.insertMany([
```

```
  {_id: 'forest', name: 'Magic Forest', description: 'Forest habitat for calm unicorns'},
```

```
  {_id: 'mountain', name: 'High Mountains', description: 'Mountain habitat for strong unicorns'},
```

```
  {_id: 'lake', name: 'Crystal Lake', description: 'Lake habitat for water-loving unicorns'}]
```

```
)
```

```
db.unicorns.updateMany(
```

```
  {name: {$in: ['Aurora', 'Nimue', 'Solnara']}},
```

```
  {$set: {habitat: {$ref: 'habitats', $id: 'forest'}}}
```

```
)
```

```
db.unicorns.updateMany(  
  {name: {$in: ['Unicrom', 'Dunx', 'Kenny']}},  
  {$set: {habitat: {$ref: 'habitats', $id: 'mountain'}}}  
)
```

```
db.unicorns.updateMany(  
  {name: {$in: ['Leia', 'Pilot']}},  
  {$set: {habitat: {$ref: 'habitats', $id: 'lake'}}}  
)
```

```
db.habitats.find()  
db.unicorns.find({habitat: {$exists: true}}, {_id: 0, name: 1, habitat: 1})
```

```
var aurora = db.unicorns.findOne({name: 'Aurora'})  
db[aurora.habitat.$ref].findOne({_id: aurora.habitat.$id})
```

```

> db.habitats.find()

db.unicorns.find(

  {habitat: {$exists: true}},

  {_id: 0, name: 1, habitat: 1}

)
< {
  name: 'Aurora',
  habitat: DBRef('habitats', 'forest')
}
{
  name: 'Unicrom',
  habitat: DBRef('habitats', 'mountain')
}
{
  name: 'Solnara',
  habitat: DBRef('habitats', 'forest')
}
{
  name: 'Kenny',
  habitat: DBRef('habitats', 'mountain')
}
{
  name: 'Leia',
  habitat: DBRef('habitats', 'lake')
}
[

```

Рисунок 30 - Создание коллекции habitats и добавление ссылок в документы unicorns

```

< {
  name: 'Aurora',
  habitat: DBRef('habitats', 'forest')
}
{
  name: 'Unicrom',
  habitat: DBRef('habitats', 'mountain')
}
{
  name: 'Solnara',
  habitat: DBRef('habitats', 'forest')
}
{
  name: 'Kenny',
  habitat: DBRef('habitats', 'mountain')
}
{
  name: 'Leia',
  habitat: DBRef('habitats', 'lake')
}
{
  name: 'Pilot',
  habitat: DBRef('habitats', 'lake')
}
{
  name: 'Nimue',
  habitat: DBRef('habitats', 'forest')
}
{
  name: 'Dunx',
  habitat: DBRef('habitats', 'mountain')
}

```

Рисунок 31 - Проверка перехода по ссылке habitat для единорога Aurora

Вывод: в документах unicorns добавлено поле habitat со ссылкой DBRef на коллекцию habitats; по ссылке можно получить документ зоны обитания.

4.27. Практическое задание 4.2.1. Создание уникального индекса по полю name

Команды для выполнения:

```
db.unicorns.createIndex({name: 1}, {unique: true})
```

```
db.unicorns.getIndexes()
```

// Проверка ограничения уникальности:

```
db.unicorns.insertOne({name: 'Aurora', loves: ['apple'], weight: 500, gender: 'f'})
```

```
db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

Рисунок 32 - Создание unique index по полю name и проверка ограничения уникальности

Вывод: уникальный индекс по name создать можно, так как имена в коллекции не повторяются; попытка добавить документ с уже существующим name завершается ошибкой duplicate key.

4.28. Практическое задание 4.3.1. Управление индексами

Команды для выполнения:

```
db.unicorns.getIndexes()
```

```
db.unicorns.dropIndexes()
```

```
db.unicorns.getIndexes()
```

// Попытка удалить обязательный индекс _id:

```
db.unicorns.dropIndex('_id_')
```

```
db.unicorns.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
> db.unicorns.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
> db.unicorns.dropIndex("name_1")

db.unicorns.getIndexes()
< [ { v: 2, key: { _id: 1 }, name: '_id_' } ]
> db.unicorns.dropIndex("_id_")
❌ > MongoServerError[InvalidOptions]: cannot drop _id index
```

Рисунок 33 - Просмотр, удаление пользовательского индекса и попытка удаления индекса _id_

```
db.numbers.find().sort({$natural: -1}).limit(4)
< {
  _id: ObjectId('6a18897b96c49528a55dd68d'),
  value: 99999
}
{
  _id: ObjectId('6a18897b96c49528a55dd68c'),
  value: 99998
}
{
  _id: ObjectId('6a18897b96c49528a55dd68b'),
  value: 99997
}
{
  _id: ObjectId('6a18897b96c49528a55dd68a'),
  value: 99996
}
```

Рисунок 34 - Создание коллекции numbers и выборка последних четырех документов

Вывод: пользовательский индекс name_1 можно удалить, но системный индекс _id_ является обязательным и не удаляется. Также была создана коллекция numbers, в которой последние документы имеют значения value 99999, 99998, 99997 и 99996.

4.29. Практическое задание 4.4.1. План запроса и сравнение индекса

Команды для выполнения:

```
db.numbers.drop()
```

```
for (let i = 0; i < 100000; i++) {
  db.numbers.insertOne({value: i})
}
```

```
db.numbers.find().sort({value: -1}).limit(4)
```



```
var noIndex = db.numbers.explain('executionStats').find({value: {$gte: 99996}}).sort({value: -1}).limit(4)
noIndex.executionStats.executionTimeMillis
```

```
// Создание индекса
```

```
db.numbers.createIndex({value: 1})
db.numbers.getIndexes()
```

```
db.numbers.find({value: {$gte: 99996}}).sort({value: -1}).limit(4)
```

```
var withIndex = db.numbers.explain('executionStats').find({value: {$gte: 99996}}).sort({value: -1}).limit(4)
withIndex.executionStats.executionTimeMillis
```

```
> db.numbers.explain("executionStats").find({value: {$gte: 99996}})
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    indexFilterSet: false,
    parsedQuery: {
      value: {
        '$gte': 99996
      }
    },
    queryHash: '8A9A1B6B',
    planCacheKey: '8A9A1B6B',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'COLLSCAN',
      filter: {
        value: {
          '$gte': 99996
        }
      },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 32,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      stage: 'COLLSCAN',
      filter: {
        value: {
          '$gte': 99996
        }
      },
      nReturned: 4,
      executionTimeMillisEstimate: 2,
      works: 100001,
      advanced: 4,
      needTime: 99996,
      needYield: 0,
      numYields: 100
    }
  }
}
```

Рисунок 35 - План выполнения запроса без индекса по полю value

```
> db.numbers.createIndex({value: 1})

db.numbers.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

Рисунок 36 - Создание индекса value и просмотр индексов коллекции numbers

```

> db.numbers.explain("executionStats").find({value: {$gte: 99996}})
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    indexFilterSet: false,
    parsedQuery: {
      value: {
        '$gte': 99996
      }
    },
    queryHash: '8A9A1B6B',
    planCacheKey: '7B918E71',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          value: 1
        },
        indexName: 'value_1',
        isMultiKey: false,
        multiKeyPaths: {
          value: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          value: [
            '[99996, inf.0]'
          ]
        }
      },
      rejectedPlans: []
    },
    executionStats: {
      executionSuccess: true,
      nReturned: 4,
      executionTimeMillis: 7,
      totalKeysExamined: 4,

```

Рисунок 37 - План выполнения запроса с индексом по полю value

```
> db.numbers.find({value: {$gte: 99996}})
< {
  _id: ObjectId('6a18897b96c49528a55dd68a'),
  value: 99996
}
{
  _id: ObjectId('6a18897b96c49528a55dd68b'),
  value: 99997
}
{
  _id: ObjectId('6a18897b96c49528a55dd68c'),
  value: 99998
}
```

Рисунок 38 - Результат выборки документов после создания индекса

Вывод: до создания индекса запрос выполнялся через последовательный просмотр коллекции: stage COLLSCAN, executionTimeMillis = 32, totalDocsExamined = 100000. После создания индекса value_1 MongoDB использовала индексный поиск: stage IXSCAN, executionTimeMillis = 7, totalKeysExamined = 4. Следовательно, индекс по полю value уменьшил количество просматриваемых элементов и ускорил выполнение запроса.

5. Выводы

В ходе выполнения лабораторной работы была создана база данных learn в СУБД MongoDB. Были выполнены операции вставки, выборки, сортировки, ограничения результата, работы с массивами, логическими операторами и проекцией полей. Также была создана коллекция towns с вложенным объектом mayor, выполнены запросы к вложенным полям, использованы курсоры, агрегированные функции count, distinct и aggregate, а также операции изменения и удаления документов. В заключительной части работы были добавлены ссылки DBRef между коллекциями, создан и проверен уникальный индекс, выполнено управление индексами и проведено сравнение плана выполнения запроса без индекса и с индексом.

6. Список источников

1. MongoDB CRUD Operations [Электронный ресурс] // MongoDB Documentation. URL: <https://docs.mongodb.com/manual/crud/>

2. MongoDB Manual [Электронный ресурс] // MongoDB Documentation. URL: <https://docs.mongodb.com/manual/>

3. Онлайн-руководство по MongoDB [Электронный ресурс] // METANIT.COM. URL: <https://metanit.com/nosql/mongodb/>

4. Кайл Б. MongoDB в действии [Электронный ресурс]. Москва: ДМК Пресс.